

E FFIT — DETAILED ALGEBRA

This appendix provides the algebraic details of the Fast Fourier Integral Transform (FFIT) as implemented in `FITFIT` (`pyfuga/trafalgar.py`). It covers the midpoint quadrature, the DFT restriction, and Bluestein's trick that rewrites the Fourier sum as a cyclic convolution.

Throughout, $\hat{f}(k)$ denotes the Fourier-space spectrum, $x_j = x_0 + a j$ are the output points with spacing a , $k_q = k_{\min} + (q + \frac{1}{2})b$ are the quadrature midpoints with spacing $b = (k_{\max} - k_{\min})/n$, and $k_0 = k_{\min} + b/2$ is the first midpoint.

E.1 Midpoint quadrature

The Fourier integral is truncated to $[k_{\min}, k_{\max}]$ and approximated by the midpoint rule. Replacing $\hat{f}(k)$ by its value at each sub-interval midpoint k_q gives

$$\int_{k_{\min}}^{k_{\max}} \hat{f}(k) e^{ikx_j} dk \approx b \sum_{q=0}^{n-1} e^{ik_q x_j} \hat{f}(k_q). \quad (\text{E.1})$$

This is a good approximation whenever $\hat{f}(k)$ is slowly varying compared to the oscillation of e^{ikx} .

E.2 The DFT restriction

Substituting $k_q = k_0 + qb$ and $x_j = x_0 + a j$ into the exponent of the sum yields

$$k_q x_j = k_0 x_0 + k_0 a j + q b x_0 + a b q j. \quad (\text{E.2})$$

If $ab = 2\pi/n$, the sum reduces to a standard DFT and can be evaluated with a single FFT. This condition is a severe restriction: we want a small enough for accurate interpolation of the output table and b small enough for the midpoint quadrature to be accurate. The only way to satisfy both with the DFT constraint is to make n very large, which may be prohibitively expensive.

E.3 Bluestein's trick: the convolution form

An FFT-based method exists that avoids the DFT restriction by exploiting a trick due to Bailey and Swarztrauber (1991); see also Bailey and Swarztrauber (1994) and Inverarity (2002). The key identity is

$$q j = \frac{1}{2} (q^2 + j^2 - (q - j)^2). \quad (\text{E.3})$$

Substituting (E.3) into (E.2) and collecting terms, we can split the exponential in the sum into three factors: one that depends only on q (the *pre-multiply phase* ϕ), one that depends only on j (the *post-multiply phase* ψ), and one that depends on the difference $j - q$ (the convolution kernel h).

Pre-multiply phase (applied before the convolution).

$$\phi(q) = e^{i \left[(k_0 + bq) x_0 + \frac{1}{2} ab q^2 \right]}. \quad (\text{E.4})$$

Post-multiply phase (applied after the convolution).

$$\psi(j) = e^{i \left[k_0 x_j + \frac{1}{2} ab j^2 \right]}. \quad (\text{E.5})$$

Convolution kernel.

$$h_j = \begin{cases} e^{-\frac{1}{2} i ab j^2}, & 0 \leq j < n, \\ e^{-\frac{1}{2} i ab (2n-j)^2}, & n \leq j < 2n. \end{cases} \quad (\text{E.6})$$

Convolution vector. The spectral field is multiplied by the pre-multiply phase and zero-padded:

$$g_q = \begin{cases} \hat{f}(k_q) \phi(q), & 0 \leq q < n, \\ 0, & n \leq q < 2n. \end{cases} \quad (\text{E.7})$$

Putting everything together, the FFIT result is

$$f(x_j) = \psi(j) (g * h)(j), \quad 0 \leq j \leq n-1, \quad (\text{E.8})$$

where $*$ denotes discrete circular convolution of period $2n$.

E.4 Algorithmic steps

The evaluation of (E.8) proceeds in three steps:

1. **Pre-multiply:** form $g_q = \hat{f}(k_q) \phi(q)$, zero-padded to length $2n$.
2. **Convolve:** compute $(g * h)(j)$ via the convolution theorem — FFT both g and h , multiply pointwise, inverse FFT. The FFT of h depends only on the grid parameters (a, b, n) and is computed once and stored.
3. **Post-multiply:** $f(x_j) = \psi(j) (g * h)(j)$.

Compared to the simple FFT method, $2n$ points are needed instead of n , and two FFTs are needed instead of one; the method is therefore about four times slower than FFT, but that is still incredibly fast. The freedom to choose a and b independently is well worth the extra cost.

E.5 Extension to 2D

The 2D version follows by separability: a 1D FFIT is applied first along x and then along y . The spectral spacings b_x and b_y and the spatial spacings $a_x = \Delta x$ and $a_y = \Delta y$ can all be chosen independently. This is implemented in the function `FITFIT` in `pyfuga/trafalgar.py`.

E.6 Connection to the implementation

The table below summarises how each mathematical symbol maps to the code in `trafalgar.py`:

Equation	Symbol	Code variable
Spectral spacing	$b_x = \pi / (n_x \Delta x) / N N N_x$	<code>bx</code>
Spectral spacing	$b_y = \pi / (n_y \Delta y) / N N N_y$	<code>by</code>
Product	$\alpha_x = \Delta x \cdot b_x$	<code>abx</code>
Product	$\alpha_y = \Delta y \cdot b_y$	<code>aby</code>
Pre-multiply (E.4)	$\phi_x(q)$	<code>phix</code>
Post-multiply (E.5)	$\psi_x(j)$	<code>psix</code>
Kernel (E.6)	$\hat{h}_x$	<code>hhatxList</code>
Convolution vector (E.7)	g	<code>ghatx</code>
FFIT result (E.8)	$f(x_j)$	<code>field</code>

Sign convention. The parameter `sign` (+1 or -1) selects the real or imaginary part of the FFIT output for each flow variable, encoding the symmetry of the spectral solution under $k \rightarrow -k$.

E.7 Approximation quality

FFIT approximates the Fourier integral as an integral over a finite interval on the k -axis. A slight improvement can be made by approximating $\hat{f}(k)$ with piecewise linear or higher-order functions between $k - b/2$ and $k + b/2$, as described in Press et al. (1992), leading to a modified attenuation function. More elaborate integration schemes have also been suggested, e.g. Simpson's rule (Simonen and Olkkonen, 1985). FFIT can be used for the inverse transformation, but unlike FFT, we do not return to the starting point when transforming back and forth. Worst of all is the lack of error estimates, which means there is little guidance in choosing a , b , and n .

Recommended spatial resolution. Given the wavenumber discretisation used in the Fourier LUTs, the recommended output spacings are

$$\Delta x = 20 \text{ m}, \quad \Delta y = 5 \text{ m}. \quad (\text{E.9})$$

The finer spacing in y reflects the sharper lateral gradients of the wake deficit, which require higher spatial resolution to resolve accurately.

REFERENCES

- Bailey, D. H. and Swarztrauber, P. N. (1991). The fractional fourier transform and applications. *SIAM Review*, 33:389–404.
- Bailey, D. H. and Swarztrauber, P. N. (1994). A fast method for the numerical evaluation of continuous fourier and laplace transforms. *SIAM J. Sci. Comput.*, 15:1105–1110.
- Inverarity, G. W. (2002). Fast computation of multidimensional fourier integrals. *SIAM J. SCI. COMPUT.*, 24:645–651.
- Press, W. H., Teukolsky, S. A., Vetterling, W. T., and Flannery, B. P. (1992). *Numerical Recipes*. Cambridge University Press, 2nd edition.
- Simonen, P. and Olkkonen, H. (1985). Fast method for computing the fourier integral transform via simpsons numerical integration. *J. Biomed. Eng.*, 7:337–340.